Lesson 4

Edges and Contours

Contents

1. Image Gradients
2. Edge Detection
3. Contours

Useful Links

- OpenCV Documentation: https://docs.opencv.org/4.x/
- OpenCV Python Tutorials: https://docs.opencv.org/4.x/d6/d00/tutorial_py_root.html
- Numpy References: https://numpy.org/doc/1.23/reference/index.html#reference

Overview

Edge and contour detection is the process of finding the outline of objects/bodies within an image. Edge finding allows the amount of data within an image to be significantly cut down while preserving information we are interested in, such as the outline of objects. By reducing unwanted data in an image, the amount of time taken to process an image can be drastically decreased.

Often we're unable to develop a tight colour threshold hold for certain objects, causing image thresholding to be an inefficient and noise-susceptible process. If we are trying to train a model

to detect dogs in an image, we must consider dogs over a wide range of colours. To combat this, we may choose to train a model to search for the outline of a dog.

Image Gradients

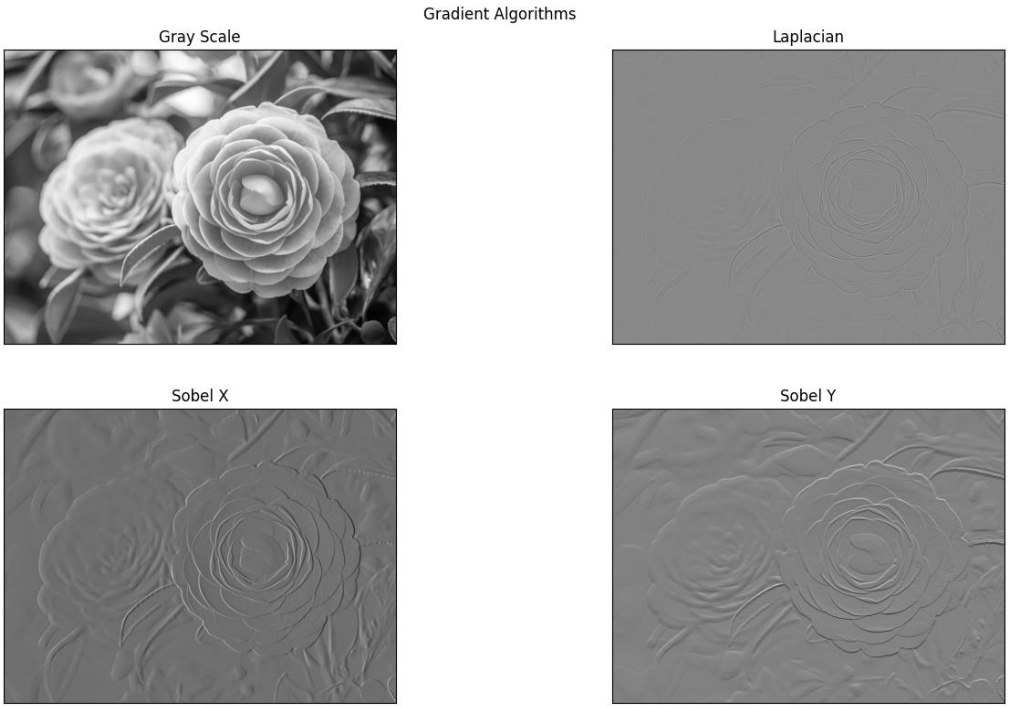
Calculating image gradients involves finding the gradient at any point across a 2D plane (the image). This process is explored in multivariable calculus and involves finding the differential in both the x and y directions. It is not required to understand the mathematics behind image gradients to be able to use edge detection, however, it is good to understand its general process and how it is applied to find image edges.

The image gradient refers to the rate of change in intensity between two pixels. A higher image gradient is a larger change in intensity (i.e. red to green) while a lower image gradient is a smaller change in intensity (i.e. red to pink).

Edge detection algorithms search for sudden changes in pixel intensity (high gradients) that may constitute an edge, for example, the gradient between the white coloured outline of the dog on the previous page compared to the green grass in the background would be high, in contrast to going from the dog's white outline to its more creamy white coloured body which would have a low gradient. The relatively high gradient between white and green results in an edge being detected, as shown in the image to the right of the dog.

Different types of gradient finding methods are demonstrated below. Note that a grayscale image is used as we are interested in colour intensities when finding gradients. If you'd like to find out more about image gradients as well as their function prototypes, visit the following link:

https://docs.opencv.org/4.x/d5/d0f/tutorial_py_gradients.html



Edge Detection

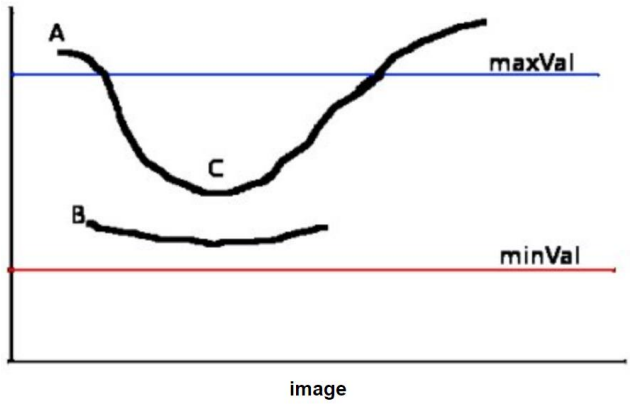
As previously discussed, edge detection algorithms make use of large changes in image gradients to find edges. One of the best and most widely used algorithms is the Canny Edge Detector. Canny receives a grayscale image as input and applies a number of operations, including a Gaussian blur and the Sobel image gradient, returning an image of edges.

Canny makes use of edge threshold values to filter out “weak edges” from “strong edges”. A stronger edge occurs when the image gradient at a particular point is larger. If the threshold values for canny are high, only stronger edges are displayed on the returned edge image. If the threshold values are lower, weaker edges are also shown. There are no set values for the Canny threshold values, with the best values to use changing depending on the image as well as what in the image you are searching for.

cv.Canny(image: cv.Mat, low: int, high: int) -> cv.Mat

- Takes a grayscale image as input.
- The low and high arguments specify the two-level threshold used for hysteresis.

What is Hysteresis?



Canny Edge Detector makes use of a clever technique known as Hysteresis to preserve weak edges we may be interested in. For an edge to be detected by Canny, it usually must be above the maxVal threshold (high Canny arg). Weaker edges such as C in the above image that fall between maxVal and minVal (low Canny arg) can also be detected as edges if and only if they are connected by two strong edges that are above maxVal. In the above image, the line containing A and C will be detected as an edge for this reason, while the line containing B will not be detected as an edge.

Edge detection algorithms are susceptible to high-frequency noise usually in the background of an image. In most cases, image smoothing using blurring must be used to reduce this noise in order to achieve optimal edge finding.

Finding Canny threshold values can be tedious and involve plenty of guesswork. Instead, use the below function which makes use of basic statistics to find reasonable min and max values.

```
def canny_threshold_values(img: cv.Mat, deviation: float=0.33) -> tuple[float,
float]:
    avgIntense = np.median(img)

    minVal = avgIntense * (1 - deviation)
    maxVal = avgIntense * (1 + deviation)

    return minVal, maxVal
```

Task 1: Finding a Dog's edges

1. Create a new python file and download the image of the dog below.

2. Convert the image to grayscale using *cv.cvtColor()* and apply a Gaussian blur with a kernel size (5,5) using *cv.GaussianBlur()*.

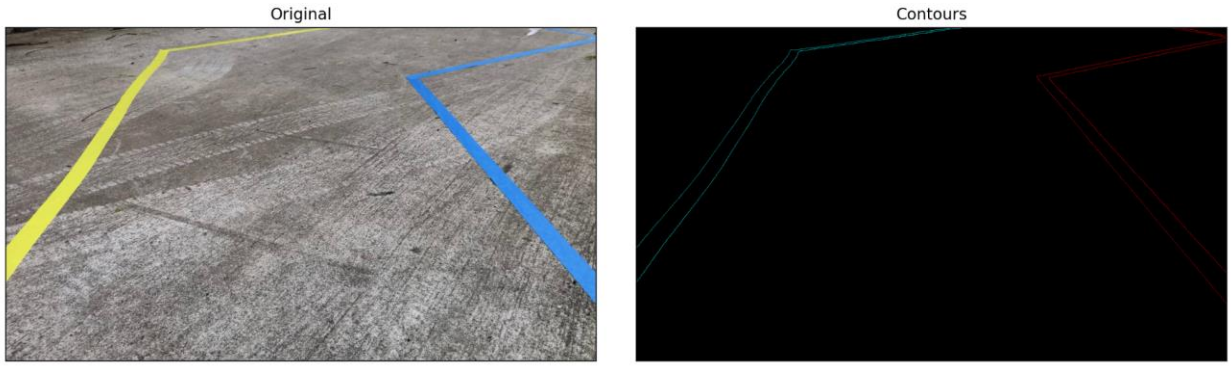
3. Pass the blurred image into *canny_threshold_values()* to retrieve the min and max values for the canny threshold.

4. Use *cv.Canny()* to detect the image edges. Display your result.

5. Increase/decrease the kernel size of the Gaussian Blur and see what happens.

Contours

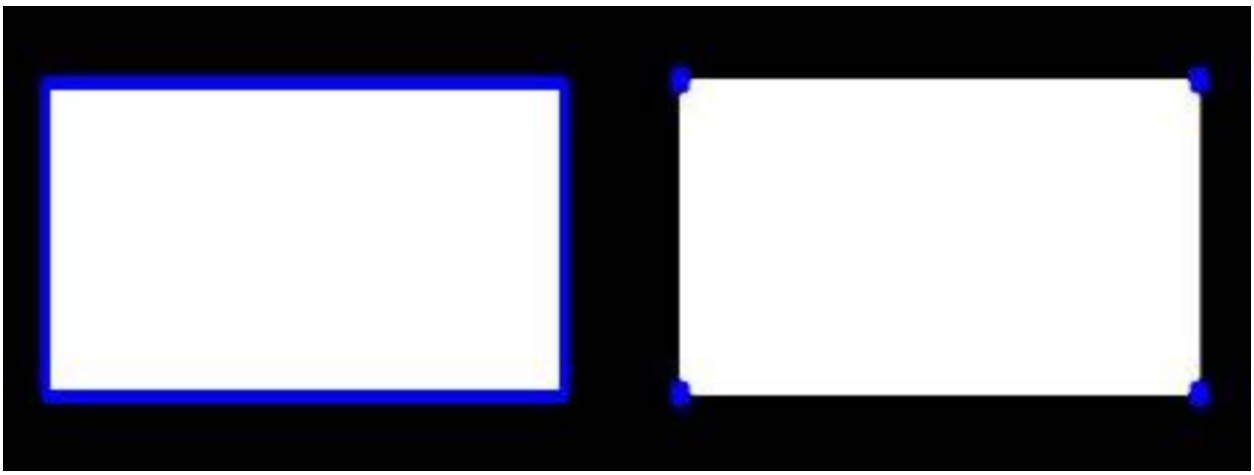
Contours are a collection of edges that enclose an area. The objects we are searching for are usually enclosed by their outline, making contours a useful tool for detection. Contours are generated of a binary image created using thresholding or edge detection. Contour and edge images often look extremely similar if not the same. Despite this, it is important to keep in mind that contours and edges are not the same and should not be used in place of each other.



cv.findContours(image: cv.Mat, mode: int, method: int) -> np.ndarray

- Returns an array of contours found in the image. Each contour consists of an array of points that define the boundary of the contour.
- Pass a binary image as the image argument.
- The mode argument specifies the format of the returned contour list. For now, use *cv.RETR_TREE* for mode. Other modes can be found here: https://docs.opencv.org/4.x/d3/dc0/group__imgproc__shape.html#ga819779b9857cc2f8601e6526a3a5bc71
- Mode determines how the contours will be retrieved from the image. Passing *cv.CHAIN_APPROX_NONE* causes every point in a contour to be returned while *cv.CHAIN_APPROX_SIMPLE* only returns the critical points in a contour, such as the contour corners, saving space. A representation of this can be seen below.

CHAIN_APPROX_NONE vs CHAIN_APPROX_SIMPLE



cv.findContours() returns two values: the contour array and the hierarchy of contours. For now, hierarchy is not important. Use *cv.findContours()* in the following manner:

```
contours, hierachy = cv.findContours(img_canny, cv.RETR_TREE, cv.CHAIN_APPROX_SIMPLE)
```

Other Contour operations

OpenCV provides us with a number of useful functions to use once we have obtained the array of contour points. These functions allow us to filter out contours we do not want or to generate bounding boxes to draw around contours.

cv.contourArea(contour) -> int

- Returns the area (in pix sq.) of a contour.
- *cv.findContours()* returns a list of contours of all contours detected in an image. Do not pass this list as the argument for *cv.contourArea()*, pass only one of the contours contained in the list at a time.
- This function is useful to filter out small contours that can result from noise. See the function below.

```
def filter_contours(min_area: int, contours: list) -> list:
    filtered_contours = []

    for contour in contours:
        if cv.contourArea(contour) > min_area:
            filtered_contours.append(contour)

    return filtered_contours
```

cv.boundingRect(contour) -> tuple

- Returns the (x,y) coordinate of the top left corner and width and height of a bounding box that surrounds a contour.
- Like *cv.contourArea()*, this function only takes a single contour as an argument.
- Use *cv.rectangle()* to draw the bounding box.

```
for contour in contours:
        x,y,w,h = cv.boundingRect(contour)
        cv.rectangle(image_copy, (x,y), (x+w, y+h), colour, 2)
```

cv.moments(contour)

- Returns a dictionary containing all the moment values of the pixels in a contour, about its centre of mass.
- Can be useful to locate the centre of mass of a contour. All pixels carry the same weight so the centre of mass is effectively the average coordinate of all the pixels contained in a contour.
- To draw the centre of mass of a contour, use the following code:

```
M = cv.moments(contour)
        if M["m00"] != 0:
            cx = int(M['m10'] / M['m00'])
            cy = int(M['m01'] / M['m00'])
            cv.circle(img, (cx,cy), 5, colour, -1)
```

Further contour operations such as contour approximation and minimum enclosing circle can be found using the following link:

https://docs.opencv.org/4.x/dd/d49/tutorial_py_contour_features.html

Task 2: Drawing a bounding box

In this task, our goal is to use contours to locate the position of a ball and draw a bounding box around it as in the image below.

Original

1. Create a new python file and download the original image of the blue ball.

2. In your program, read the image and create a grayscale copy.

3. Find the edges of the ball using Canny edge detector. Make sure to use the provided function to obtain the min and max canny threshold values.
4. Pass the canny image into *cv.findContours()* to get a list of contours in the image. You may wish to filter these contours for area, however, as the image contains very little noise as is, this step is not required.

5. Using a for loop, draw the bounding rectangle of the contours onto the original image. Find the coordinates of the bounding rectangle using *cv.boundingRect()*. If you've forgotten how to use *cv.rectangle()* you can find an explanation in lesson 2 or in the OpenCV docs.

6. Display your result using *cv.imshow()*.